МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ В ЕНЕРГЕТИЦІ

ЗВІТ З ПРАКТИЧНОЇ РОБОТИ №1

з дисципліни: «РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ МОБІЛЬНИХ ПРИСТРОЇВ» на тему:

«Атомна електростанція з радіаційним моніторингом»

Виконав студент 4 курсу групи ТВ-33
Домарацький Дмитро Олександрович

Посилання на GitHub репозиторій

dmtrDO/mobile

Київ — 2026

ПРАКТИЧНЕ ЗАНЯТТЯ № 1

Тема: Ознайомлення з Android Studio, Kotlin, Jetpack Compose. Створення першого «Hello World» застосунку.

Мета: Ознайомлення із основами розробки мобільних застосунків на платформі Android із використанням мови Kotlin.

Просунутий рівень (рисунок 1.1)

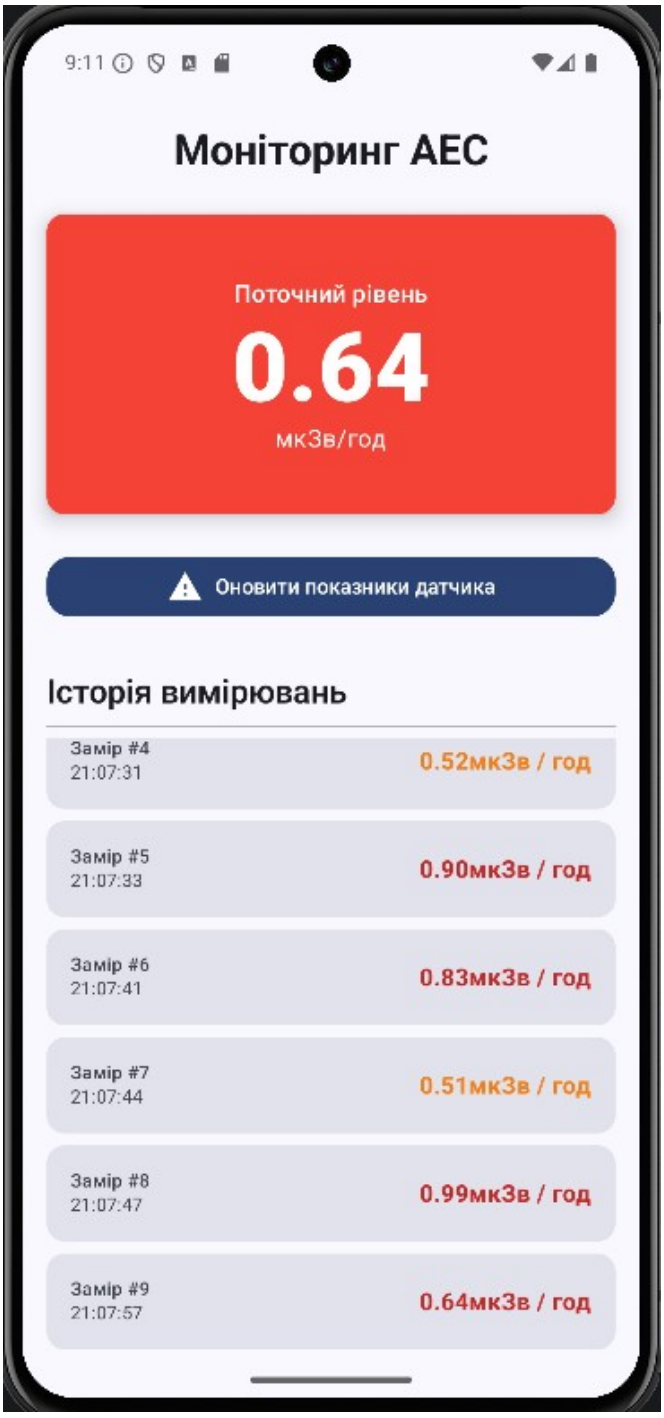


Рисунок 1.1 — Результат роботи застосунку просунутого рівня складності

Загальна ідея: проєкт є навчальним Android-застосунком, створеним для ознайомлення з Android Studio та Jetpack Compose. Його мета полягає у демонстрації принципів побудови сучасних інтерфейсів, управління станом (State) та використання анімацій для візуалізації критичних даних. Тематика застосунку пов'язана з предметною галуззю енергетики та імітує роботу системи радіаційного моніторингу на АЕС, де важлива миттєва реакція інтерфейсу на зміну показників безпеки.

Структура і компоненти: основним елементом застосунку є **MainActivity.kt** (структура проєкту зображена на рисунку 1.2), який виступає точкою входу. Для організації даних створено клас RadiationReading, що зберігає значення заміру, час та унікальний ідентифікатор. Інтерфейс розділений на логічні **Composable-функції**:

AppScreen: головний екран, що містить логіку управління станом та композицію елементів.

Card (поточний рівень): інтерактивний блок, що відображає основний показник.

HistoryItem: окремий компонент для візуалізації рядка в списку історії вимірювань. Для відображення списку використано **LazyColumn**, що забезпечує ефективне рендерення великої кількості записів.

Використані підходи UI побудований декларативно за допомогою **Jetpack Compose.** В роботі реалізовано просунутий рівень управління станом:

Динамічна зміна кольору: Використано **animateColorAsState** для плавного переходу фону картки між безпечним (зеленим), попереджувальним (жовтим) та небезпечним (червоним) рівнями.

Анімація значень: застосовано **animateFloatAsState** для візуально приємної зміни числових показників радіації.

Управління списками: використано **mutableStateListOf** для автоматичного оновлення списку історії при додаванні нових замірів. Стилізація виконана через **Material 3**, що забезпечує відповідність сучасним стандартам дизайну Android.

Взаємодія в застосунку: при запуску застосунку користувач бачить поточний рівень радіації. Головним елементом взаємодії є кнопка «Оновити показники датчика». При натисканні на неї:

1. Генерується нове випадкове значення радіаційного фону.
2. Інтерфейс плавно змінює колір та цифри.
3. Новий запис автоматично додається у верхню частину списку історії вимірювань з фіксацією точного часу. Це дозволяє користувачу не лише бачити поточний стан, а й відстежувати динаміку змін у часі.

Підсумок: застосунок має чисту архітектуру з чітким розділенням логіки та відображення. Завдяки використанню **Jetpack Compose State** та анімацій, вдалося створити наочну систему моніторингу, яка може бути розширена для інтеграції з реальними API датчиків або системами оповіщення на енергетичних об’єктах.

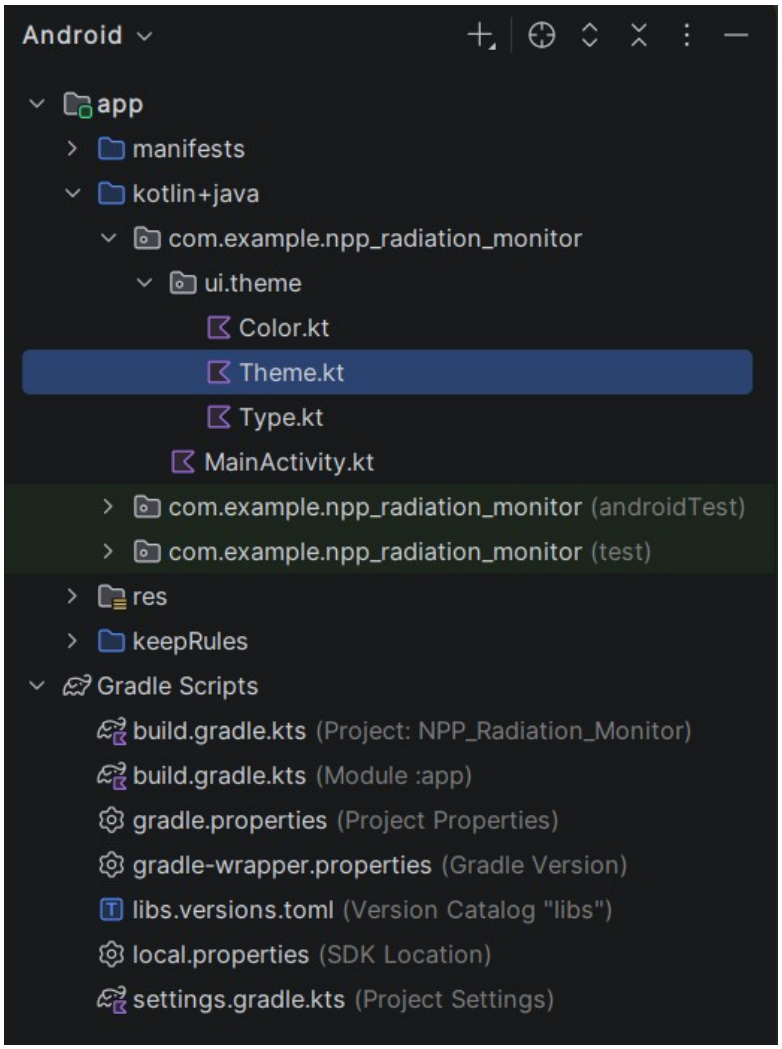


Рисунок 1.2 — Структура проєкту

ПРАКТИЧНЕ ЗАНЯТТЯ № 2

Тема: Налаштування Android-проєкту для предметної галузі енергетики. Використання Gradle та організація структури.

Мета: Сформувати практичні навички створення Android-застосунку з предметної галузі енергетики, навчитися працювати з Gradle, підключати залежності, організовувати структуру проєкту та відокремлювати логіку в окремі модулі для подальшого масштабування.

Просунутий рівень (рисунок 2.1)

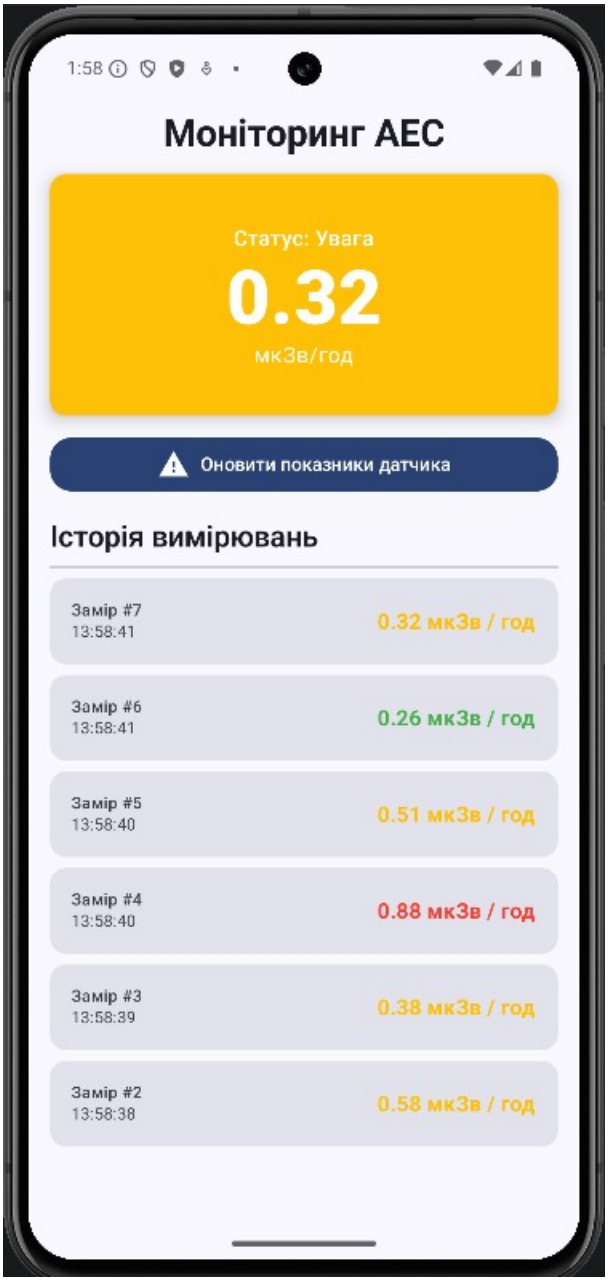


Рисунок 2.1 — Результат роботи застосунку просунутого рівня складності

Загальна ідея

Проєкт представляє Android-застосунок «Моніторинг АЕС» (NPP_Radiation_Monitor), призначений для відстеження та візуалізації радіаційного фону на об'єктах атомної енергетики в режимі реального часу (вимірювання у мкЗв/год).

Застосунок реалізовано на просунутому рівні складності, що передбачає повне винесення бізнес-логіки з файлу MainActivity в окремі модульні шари, правильну організацію Gradle-залежностей, розбиття інтерфейсу на ізольовані UI-компоненти та підготовку архітектури до майбутнього масштабування.

Структура коду та компоненти

Основною точкою входу є MainActivity, яка очищена від верстки та бізнес-логіки й відповідає лише за ініціалізацію теми NPP_Radiation_MonitorTheme та виклик головного екрана RadiationScreen.

Весь проєкт структуровано за відповідними пакетами:

Шар даних (data):

1. data.model.RadiationStatus — enum-клас категорій небезпеки (SAFE, WARNING, DANGER), який визначає текстову назву статусу, колір індикації та порогові межі радіаційного фону (до 0.30 мкЗв/год — безпечно, до 0.60 мкЗв/год — увага, понад 0.60 мкЗв/год — небезпека).
2. data.model.RadiationReading — data-клас для збереження окремого вимірювання, що містить унікальний ідентифікатор id, значення радіації value, часову мітку timestamp та автоматично розрахований status.
3. data.sensor.RadiationSensor — інтерфейс джерела даних та його реалізація SimulatedRadiationSensor, яка симулює роботу реального фізичного датчика АЕС, генеруючи показники вимірювання та форматуючи час через LocalDateTime.
4. data.RadiationManager — клас управління бізнес-логікою вимірювань, який створює початковий замір та генерує нові дані при запиті.

Шар інтерфейсу (ui):

1. ui.components.StatusCard — Composable-компонент для показу поточного стану датчика. Використовує анімації animateFloatAsState для плавної зміни чисел та animateColorAsState для анімованої зміни кольору картки залежно від рівнів небезпеки.
2. ui.components.HistoryItem — Composable-компонент для відображення окремого запису вимірювання в списку історії із зазначенням номера заміру, часу та кольорового значення радіації.
3. ui.screens.RadiationScreen — головний екран застосунку, який об'єднує всі UI-компоненти, керує станом через remember і mutableStateListOf, містить кнопку оновлення показників датчика та список історії LazyColumn.
4. ui.theme (Color.kt, Theme.kt, Type.kt) — налаштування колірної гами, типографіки та системної теми Material Design 3.

Використані підходи

Проєкт реалізований із застосуванням сучасних стандартів Android-розробки:

1. Jetpack Compose — для декларативного опису графічного інтерфейсу користувача.
2. State Management (remember, mutableStateOf, mutableStateListOf) — для збереження поточних замірів, реактивного оновлення UI та динамічного ведення історії при дії користувача.
3. Material Design 3 — для уніфікованого стилізації компонентів, анімації кольорів та чисел.
4. Винесення бізнес-логіки, моделей та симуляції датчика з MainActivity у окремі пакети (data та ui).

Взаємодія в застосунку

1. При запуску застосунку RadiationManager генерує початковий замір радіаційного фону, який передається у StatusCard.

2. Користувач натискає кнопку «Оновити показники датчика».
3. Задіюється SimulatedRadiationSensor, який зчитує нове значення та фіксує поточний час.
4. UI оновлюється автоматично завдяки Compose State: значення показника та колір картки плавно анімуються відповідно до розрахованого рівня небезпеки (SAFE, WARNING або DANGER).
5. Новий замір додається на початок списку історії (LazyColumn), де користувач може переглядати всі попередні заміри з відмітками часу.

Підсумок

Проєкт є гнучким, чисто структурованим та масштабованим (рисунок 2.2). У ньому легко замінити симулятор SimulatedRadiationSensor на реальний Bluetooth/Network API датчик радіації, підключити локальну базу даних без необхідності переписувати UI-компоненти.

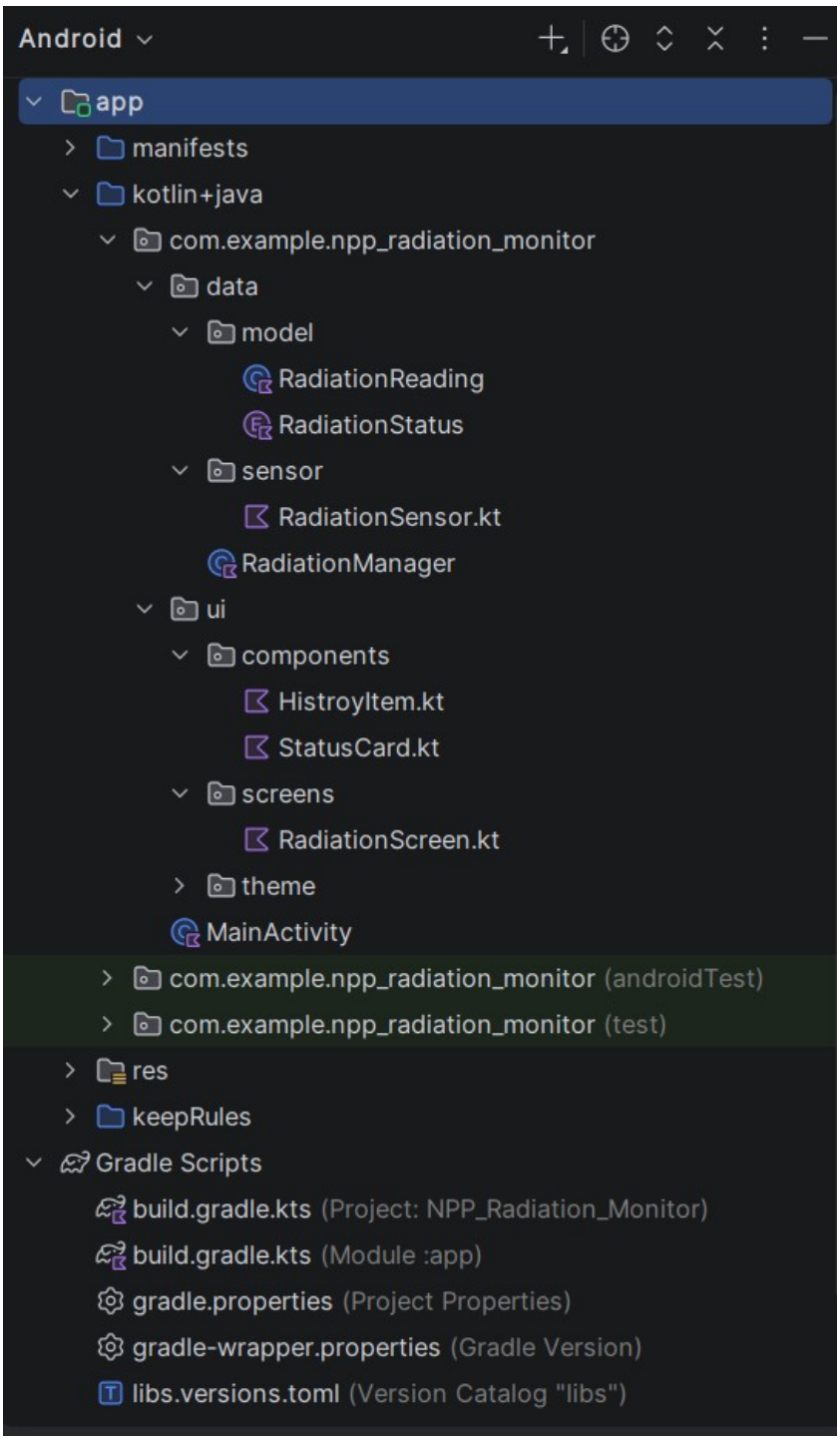


Рисунок 2.2 — Структура проєкту

ПРАКТИЧНЕ ЗАНЯТТЯ № 3

Тема: Основи Jetpack Compose: створення composable функцій, робота з layout, текстом, зображеннями та кнопками.

Мета: Ознайомлення студентів з базовими принципами роботи Jetpack Compose.

Просунутий рівень (рисунок 3.1)

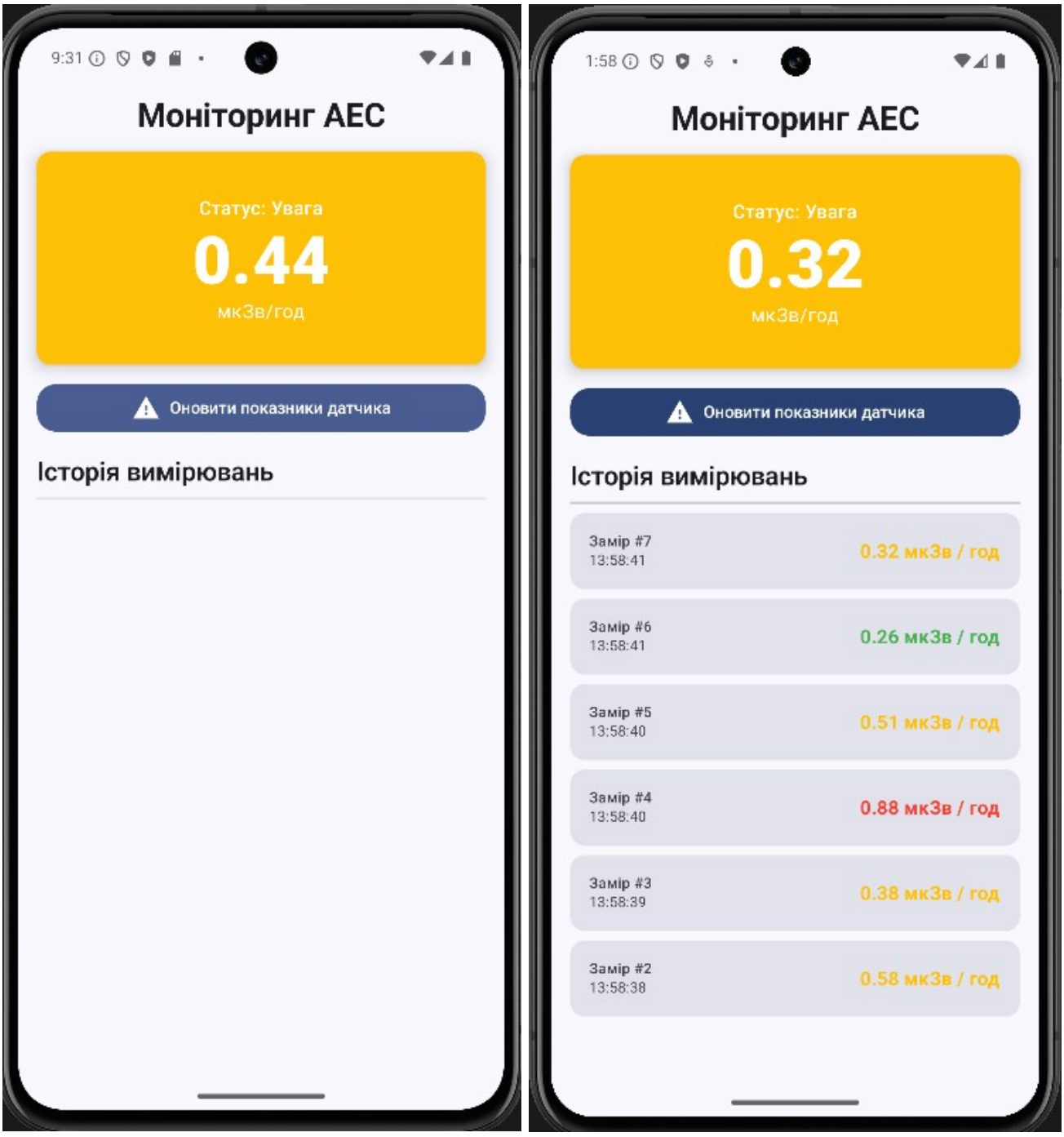


Рисунок 3.1 — Результат роботи застосунку просунутого рівня складності

Архітектура UI та Composable-функції

Інтерфейс програми розбито на окремі модульні Composable-функції, кожна з яких відповідає за відображення конкретного вузла UI:

1. RadiationScreen (головний контейнер інтерфейсу) об'єднує всі елементи екрана в єдину вертикальну структуру за допомогою компонента Column. Містить локальний стан екрана, створений за допомогою remember { mutableStateOf(...) } для збереження поточного заміру та remember { mutableStateListOf(...) } для динамічного списку історії. Організовує відступи та центрування елементів через модифікатори Modifier.fillMaxSize(), Modifier.padding(16.dp) та horizontalAlignment = Alignment.CenterHorizontally.
2. StatusCard (інформаційний блок поточного стану) побудований на основі компонента Card із закругленими кутами та висотою elevation 8.dp. Всередині використовує Column з вертикальним та горизонтальним вирівнюванням по центру (Arrangement.Center, Alignment.CenterHorizontally). Включає три текстові елементи Text: заголовок статусу, числове значення випромінювання (збільшений шрифт 56.sp та FontWeight.Black) та одиниці вимірювання ("мкЗв/год"). Реалізує плавну анімацію зміни фону та числових значень при надходженні нових даних.
3. HistoryItem (елемент списку вимірювань) — окремий Composable-компонент для відображення картки з фоновим кольором surfaceVariant. Для горизонтального розміщення даних використовується контейнер Row з властивістю Arrangement.SpaceBetween, що розносить текстовий блок з номером/часом заміру ліворуч, а числове значення радіації — праворуч.

Робота з Layout, Текстом, Кнопками та Modifier

Під час розробки UI-шару застосовано наступні інструменти та компоненти Jetpack Compose:

1. Column — використовується для послідовного розміщення заголовка, картки стану, кнопки керування та розділу історії зверху вниз.
2. Row — застосовується всередині кнопки для вирівнювання іконки поруч із текстом, а також у картці історії для горизонтального розподілу інформації.
3. LazyColumn — оптимізований список для відтворення історії замірів із підтримкою відступів між елементами через verticalArrangement = Arrangement.spacedBy(8.dp).
4. Spacer та HorizontalDivider — забезпечують фіксовані вертикальні інтервали та візуальні розділювальні лінії між блоками UI.
5. Численні показники відформатовано до двох знаків після коми за допомогою String.format(locale = Locale.UK, "%.2f", ...).
6. Елемент керування реалізовано за допомогою Button із закругленою формою MaterialTheme.shapes.large та можливістю розтягування на всю ширину Modifier.fillMaxWidth(). Всередину кнопки інтегровано векторну іконку Icon(Icons.Default.Warning, contentDescription = null) та текстовий підпис, розділені горизонтальним відступом Spacer(Modifier.width(8.dp)).
7. Усі компоненти налаштовано через ланцюжки модифікаторів: задано внутрішні та зовнішні відступи (padding), точні або адаптивні розміри (fillMaxSize, fillMaxWidth, height(180.dp)), що забезпечує коректний вигляд на пристроях з різною роздільною здатністю екрана.

Механізм взаємодії

Початковий стан: Під час первинного відображення Composable-функція RadiationScreen ініціалізує початковий замір радіації через RadiationManager та відображає його у StatusCard. При натисканні на кнопку «Оновити показники датчика» спрацьовує лямбда-функція onClick, яка викликає генерацію нового заміру в об'єкті RadiationManager. Стан currentReading.value оновлюється новим об'єктом RadiationReading, а у список history додається новий запис. Jetpack Compose виявляє зміну стану (State Change) і автоматично виконує повторний

рендеринг тільки тих Composable-функцій, які залежать від цих даних (StatusCard та LazyColumn), забезпечуючи швидкий та плавній UI-відгук.

Підсумок

Виконане завдання повністю демонструє можливості Jetpack Compose щодо створення сучасних декларативних інтерфейсів (рисунок 3.2). Завдяки правильному комбінуванню layout-компонентів (Column, Row, LazyColumn), гнучкому налаштуванню через Modifier, використанню кнопок, іконок, карт та реактивних станів (remember, mutableStateOf), вдалося побудувати масштабовану та адаптивну систему відображення радіаційного моніторингу.

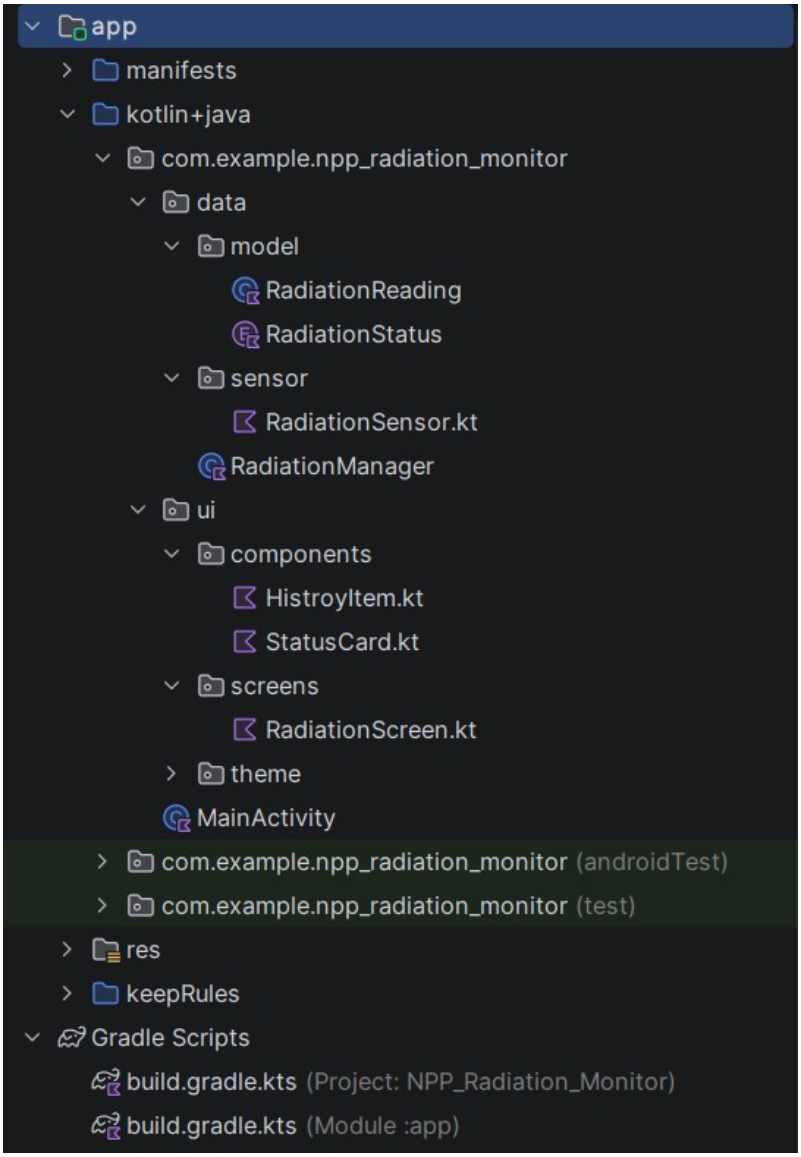


Рисунок 3.2 — Структура проєкту

ПРАКТИЧНЕ ЗАНЯТТЯ № 4

Тема: Створення екрану авторизації та реєстрації користувача в мобільному застосунку.

Мета: Ознайомлення з принципами побудови UI/UX для екранів входу та реєстрації.

Просунутий рівень (рисунок 4.1)

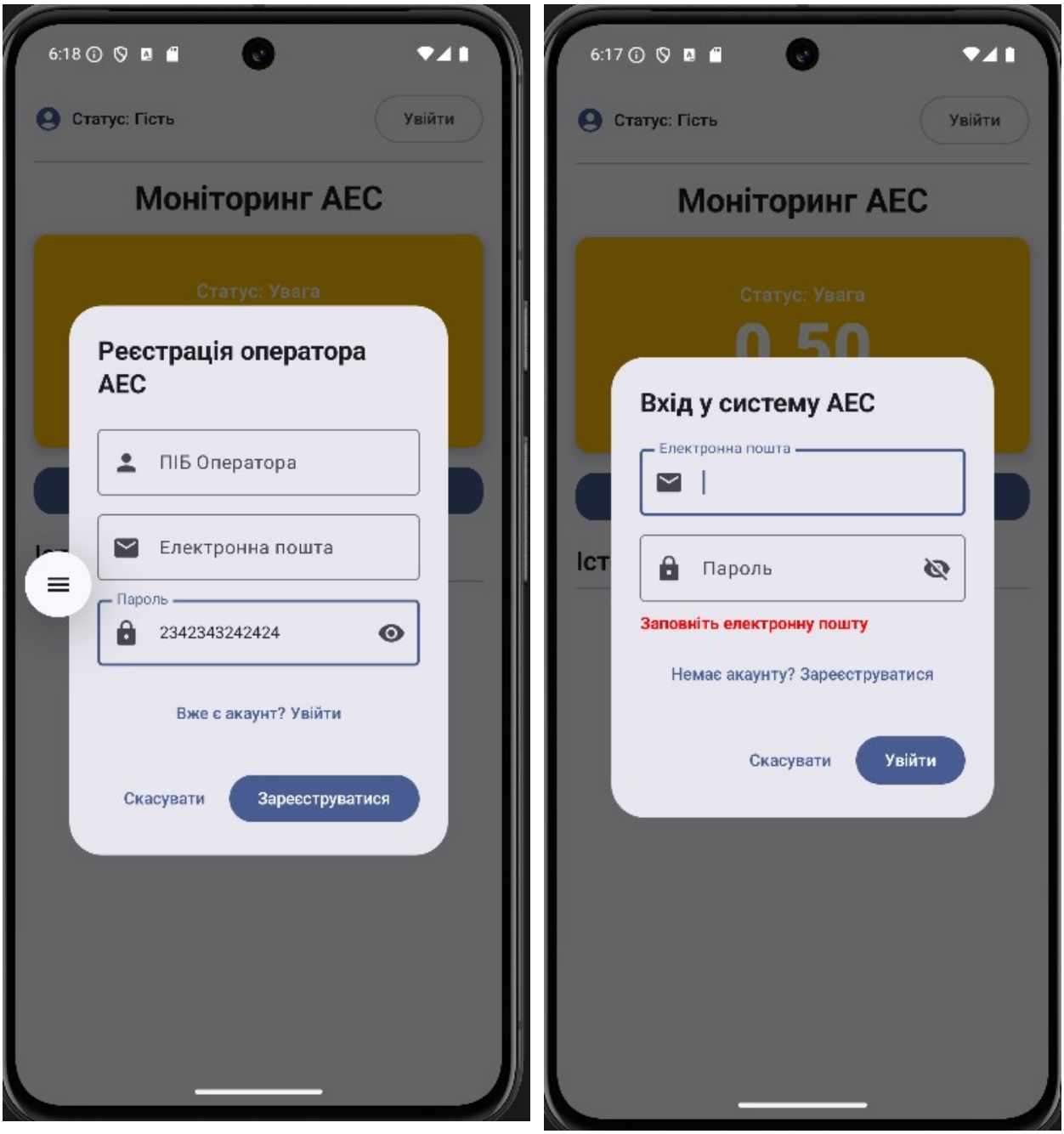


Рисунок 4.1 — Результат роботи застосунку просунутого рівня складності

Загальна ідея

Застосунок демонструє принципи побудови сучасних мобільних UI за допомогою Jetpack Compose, використання спливаючих модальних вікон (AlertDialog), обробку стану форми, динамічну зміну режимів «Вхід» / «Реєстрація» та відображення повідомлень про помилки введення даних.

Структура і компоненти

Головним екраном застосунку є RadiationScreen, у верхній частині якого інтегровано шапку стану профілю (статус «Гість» або «Оператор: [email/name]») з кнопками «Увійти» або «Вийти».

Основні Composable-функції та модулі включають:

1. AuthDialog (ui/components/AuthDialog.kt) — основний Composable-компонент спливаючого вікна модального типу (AlertDialog), що забезпечує форму авторизації та реєстрації інженера АЕС.
2. OutlinedTextField — текстові поля з зовнішніми рамками та іконками (Person, Email, Lock) для введення ПІБ оператора, електронної пошти та пароля. Для поля пароля реалізовано інтерактивну кнопку IconButton з іконками Visibility та VisibilityOff для включення/виключення маскування пароля через PasswordVisualTransformation.
3. Errors (ui/components/AuthDialog.kt) — окрема Composable-функція для візуалізації червоного тексту помилки валідації.
4. checkErrors — бізнес-функція перевірки введених даних, що перевіряє поштовий адрес на заповненість та пароль на мінімальну довжину (не менше 6 символів).
5. Button та TextButton — кнопки підтвердження дії («Увійти» / «Зареєструватися») та перемикання між режимами входу й реєстрації всередині вікна.

Використані підходи

У проєкті застосовано Jetpack Compose для декларативного створення реактивного UI та управління станом через remember і mutableStateOf:

1. State Management: змінні стану isRegisterMode (перемикач режиму входу/реєстрації), email, password, name, passwordVisible (видимість пароля) та showError (прапорець показу помилки).
2. Material Design 3: стилізація текстових полів, кнопок та колірної індикації помилок.
3. Модульність та спливаючі модальні вікна: використання AlertDialog дозволяє користувачеві проходити авторизацію прямо поверх основного екрана моніторингу АЕС без втрати контексту.
4. UX-оптимізація валідації: при редагуванні будь-якого поля (onValueChange) прапорець помилки showError автоматично скидається у false, що запобігає передчасному появі нерелевантних підказок.

Взаємодія в застосунку

При запуску застосунку відкривається екран моніторингу АЕС із кнопкою «Увійти» у верхній шапці. При натисканні кнопки з'являється спливаюче вікно AuthDialog у режимі входу. За потреби користувач може переключитися на форму реєстрації натисканням текстової кнопки. Користувач вводить пошту та пароль (у режимі реєстрації також додається поле ПІБ). При натисканні кнопки «Увійти» / «Зареєструватися» викликається функція checkErrors: якщо дані некоректні — вмикається showError = true і відображається текст помилки червоним кольором. Якщо дані правильні — вікно закривається, а у шапці головного екрана стан змінюється з «Гість» на «Оператор: [Ім'я/Email]» та з'являється кнопка «Вийти». Натискання кнопки «Вийти» скидає стан авторизації назад до статусу «Гість».

Підсумок

Проєкт демонструє гнучкість використання Jetpack Compose для створення інтерактивних форм введення та управління станом (рисунок 4.2). Використання спливаючого вікна AlertDialog та винесення валідації й відображення помилок в окремі функції забезпечує модульність та читабельність коду.

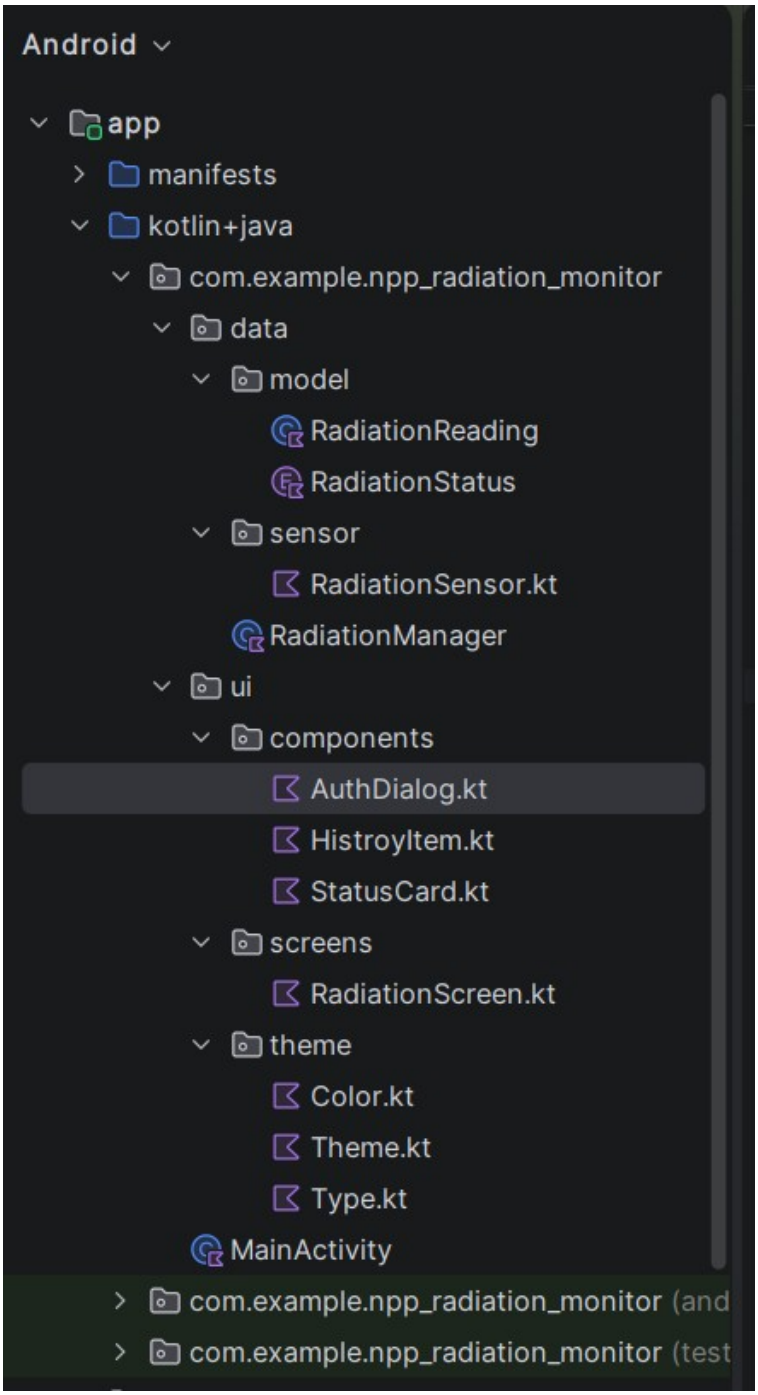


Рисунок 4.2 — Структура проєкту

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №1

Тека com/example/npp_radiation_monitor/

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor

import android.os.Build
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.annotation.RequiresApi
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableFloatStateOf
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.unit.sp
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
import java.util.Locale
import kotlin.random.Random

class MainActivity : ComponentActivity() {
    @RequiresApi(Build.VERSION_CODES.O)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Surface { AppScreen() }
            }
        }
    }
}

// Клас для зберігання даних про вимірювання
data class RadiationReading(
    val id: Int,
    val value: Float,
    val timestamp: String
)

@RequiresApi(Build.VERSION_CODES.O)
@Composable
fun AppScreen() {
    // Стан для поточного значення радіації
    var currentRadiation by remember { mutableFloatStateOf(0.12f) }
    // Список історії вимірювань
    val history = remember { mutableStateListOf<RadiationReading>() }
    // Лічильник для ID
    var nextId by remember { mutableIntStateOf(1) }

    // Анімація числа
    val animatedRadiation by animateFloatAsState(targetValue =
currentRadiation)

    // Логіка вибору кольору залежно від рівня
    val statusColor by animateColorAsState(
        targetValue = when {
            currentRadiation < 0.3f -> Color(0xFF4CAF50) // Зелений
(Безпечно)
            currentRadiation < 0.6f -> Color(0xFFFFC107) // Жовтий
(Увага)
            else -> Color(0xFFF44336) // Червоний (Небезпека)
        },
```

```
    )

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 50.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Моніторинг АЕС",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 24.dp)
        )

        // Картка поточного стану
        Card(
            modifier = Modifier
                .fillMaxWidth()
                .height(200.dp),
            colors = CardDefaults.cardColors(containerColor =
statusColor),
            elevation = CardDefaults.cardElevation(defaultElevation =
8.dp)
        ) {
            Column(
                modifier = Modifier.fillMaxSize(),
                verticalArrangement = Arrangement.Center,
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(
                    text = "Поточний рівень",
                    color = Color.White,
                    style = MaterialTheme.typography.titleMedium
                )
                Text(
                    text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                    color = Color.White,
                    fontSize = 64.sp,
                    fontWeight = FontWeight.Black
                )
                Text(
                    text = "мкЗв/год",
                    color = Color.White,
                    style = MaterialTheme.typography.bodyLarge
                )
            }
        }
```

```
        Spacer(modifier = Modifier.height(24.dp))

        // Кнопка оновлення
        Button(
            onClick = {
                val newValue = Random.nextFloat() * 1.0f // Випадкове
значення від 0 до 1.0
                currentRadiation = newValue
                val time =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
                history.add(0, RadiationReading(nextId++, newValue,
time))
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(32.dp))

        // Заголовок історії
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
            Text(
                text = "Історія вимірювань",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        // Список історії
        LazyColumn(
            modifier = Modifier.fillMaxWidth(),
            contentPadding = PaddingValues(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history) { reading -> HistoryItem(reading) }
        }
    }
}

@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
```

```
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold,
                color = when {
                    reading.value < 0.3f -> Color(0xFF2E7D32)
                    reading.value < 0.6f -> Color(0xFFF57F17)
                    else -> Color(0xFFC62828)
                }
            )
        }
    }
}
```

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №2

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationReading

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
    val id: Int,
    val value: Float,
    val timestamp: String,
    val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationStatus

```
package com.example.npp_radiation_monitor.data.model

import androidx.compose.ui.graphics.Color

enum class RadiationStatus(
 val title: String,
 val color: Color,
 val maxThreshold: Float
) {
 SAFE("Безпечно", Color(0xFF4CAF50), 0.30f),
 WARNING("Увага", Color(0xFFFFC107), 0.60f),
 DANGER("Небезпека", Color(0xFFF44336), Float.MAX_VALUE);

 companion object {
 fun fromValue(value: Float): RadiationStatus {
 return when {
 value < SAFE.maxThreshold -> SAFE
 value < WARNING.maxThreshold -> WARNING
 else -> DANGER
 }
 }
 }
}
```

Тека com/example/npp_radiation_monitor/data/sensor

Файл RadiationSensor.kt

```
package com.example.npp_radiation_monitor.data.sensor
import android.os.Build
import androidx.annotation.RequiresApi
import com.example.npp_radiation_monitor.data.model.RadiationReading
```

```
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
import kotlin.random.Random

interface RadiationSensor {
    fun getReading(id: Int): RadiationReading
}

class SimulatedRadiationSensor : RadiationSensor {
    @RequiresApi(Build.VERSION_CODES.O)
    override fun getReading(id: Int): RadiationReading {
        val value = Random.nextFloat() * 1.0f // Значення від 0.00 до
1.00 мкЗв/год
        val time =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
        return RadiationReading(
            id = id,
            value = value,
            timestamp = time
        )
    }
}
```

Тека com/example/npp_radiation_monitor/data

Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor

class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 1

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл HistroyItem.kt

```
package com.example.npp_radiation_monitor.ui.components
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold,
                color = reading.status.color
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл StatusCard.kt

```
package com.example.npp_radiation_monitor.ui.components
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

@Composable
fun StatusCard(
    reading: RadiationReading,
    modifier: Modifier = Modifier
) {
    val animatedRadiation by animateFloatAsState(targetValue =
reading.value)
    val animatedColor by animateColorAsState(targetValue =
reading.status.color)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .height(180.dp),
        colors = CardDefaults.cardColors(containerColor = animatedColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "Статус: ${reading.status.title}",
                color = Color.White,
```

```
                style = MaterialTheme.typography.titleMedium
            )
            Text(
                text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                color = Color.White,
                fontSize = 56.sp,
                fontWeight = FontWeight.Black
            )
            Text(
                text = "мкЗв/год",
                color = Color.White,
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
```

```
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.ui.components.HistoryItem
import com.example.npp_radiation_monitor.ui.components.StatusCard

@Composable
fun RadiationScreen(
    radiationManager: RadiationManager = remember { RadiationManager() }
) {
    var currentReading by remember
{ mutableStateOf(radiationManager.createInitialReading()) }
    val history = remember { mutableStateListOf<RadiationReading>() }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Моніторинг АЕС",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // Картка поточного заміру
        StatusCard(reading = currentReading)

        Spacer(modifier = Modifier.height(12.dp))

        // Кнопка оновлення
        Button(
            onClick = {
                val newReading = radiationManager.generateNextReading()
                currentReading = newReading
                history.add(0, newReading)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Історія
```

```
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
            Text(
                text = "Історія вимірювань",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        LazyColumn(
            modifier = Modifier.fillMaxWidth(),
            contentPadding = PaddingValues(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history, key = { it.id }) { reading ->
                HistoryItem(reading = reading)
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.material3.Surface
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Surface {
                    RadiationScreen()
                }
            }
        }
    }
}
```

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №3

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationReading

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
    val id: Int,
    val value: Float,
    val timestamp: String,
    val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationStatus

```
package com.example.npp_radiation_monitor.data.model

import androidx.compose.ui.graphics.Color

enum class RadiationStatus(
 val title: String,
 val color: Color,
 val maxThreshold: Float
) {
 SAFE("Безпечно", Color(0xFF4CAF50), 0.30f),
 WARNING("Увага", Color(0xFFFFC107), 0.60f),
 DANGER("Небезпека", Color(0xFFF44336), Float.MAX_VALUE);

 companion object {
 fun fromValue(value: Float): RadiationStatus {
 return when {
 value < SAFE.maxThreshold -> SAFE
 value < WARNING.maxThreshold -> WARNING
 else -> DANGER
 }
 }
 }
}
```

Тека com/example/npp_radiation_monitor/data/sensor

Файл RadiationSensor.kt

```
package com.example.npp_radiation_monitor.data.sensor
import android.os.Build
import androidx.annotation.RequiresApi
import com.example.npp_radiation_monitor.data.model.RadiationReading
```

```
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
import kotlin.random.Random

interface RadiationSensor {
    fun getReading(id: Int): RadiationReading
}

class SimulatedRadiationSensor : RadiationSensor {
    @RequiresApi(Build.VERSION_CODES.O)
    override fun getReading(id: Int): RadiationReading {
        val value = Random.nextFloat() * 1.0f // Значення від 0.00 до
1.00 мкЗв/год
        val time =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
        return RadiationReading(
            id = id,
            value = value,
            timestamp = time
        )
    }
}
```

Тека com/example/npp_radiation_monitor/data

Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor

class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 1

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл HistroyItem.kt

```
package com.example.npp_radiation_monitor.ui.components
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold,
                color = reading.status.color
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл StatusCard.kt

```
package com.example.npp_radiation_monitor.ui.components
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

@Composable
fun StatusCard(
    reading: RadiationReading,
    modifier: Modifier = Modifier
) {
    val animatedRadiation by animateFloatAsState(targetValue =
reading.value)
    val animatedColor by animateColorAsState(targetValue =
reading.status.color)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .height(180.dp),
        colors = CardDefaults.cardColors(containerColor = animatedColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "Статус: ${reading.status.title}",
                color = Color.White,
```

```
                style = MaterialTheme.typography.titleMedium
            )
            Text(
                text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                color = Color.White,
                fontSize = 56.sp,
                fontWeight = FontWeight.Black
            )
            Text(
                text = "мкЗв/год",
                color = Color.White,
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
```

```
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.ui.components.HistoryItem
import com.example.npp_radiation_monitor.ui.components.StatusCard

@Composable
fun RadiationScreen(
    radiationManager: RadiationManager = remember { RadiationManager() }
) {
    var currentReading by remember
{ mutableStateOf(radiationManager.createInitialReading()) }
    val history = remember { mutableStateListOf<RadiationReading>() }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Моніторинг АЕС",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // Картка поточного заміру
        StatusCard(reading = currentReading)

        Spacer(modifier = Modifier.height(12.dp))

        // Кнопка оновлення
        Button(
            onClick = {
                val newReading = radiationManager.generateNextReading()
                currentReading = newReading
                history.add(0, newReading)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Історія
```

```
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
            Text(
                text = "Історія вимірювань",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        LazyColumn(
            modifier = Modifier.fillMaxWidth(),
            contentPadding = PaddingValues(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history, key = { it.id }) { reading ->
                HistoryItem(reading = reading)
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.material3.Surface
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            NPP_Radiation_MonitorTheme {
                Surface {
                    RadiationScreen()
                }
            }
        }
    }
}
```

ПРОГРАМНИЙ КОД ПРАКТИЧНОГО ЗАНЯТТЯ №4

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationReading

```
package com.example.npp_radiation_monitor.data.model

data class RadiationReading(
    val id: Int,
    val value: Float,
    val timestamp: String,
    val status: RadiationStatus = RadiationStatus.fromValue(value)
)
```

Тека com/example/npp_radiation_monitor/data/model

Файл RadiationStatus

```
package com.example.npp_radiation_monitor.data.model

import androidx.compose.ui.graphics.Color

enum class RadiationStatus(
 val title: String,
 val color: Color,
 val maxThreshold: Float
) {
 SAFE("Безпечно", Color(0xFF4CAF50), 0.30f),
 WARNING("Увага", Color(0xFFFFC107), 0.60f),
 DANGER("Небезпека", Color(0xFFF44336), Float.MAX_VALUE);

 companion object {
 fun fromValue(value: Float): RadiationStatus {
 return when {
 value < SAFE.maxThreshold -> SAFE
 value < WARNING.maxThreshold -> WARNING
 else -> DANGER
 }
 }
 }
}
```

Тека com/example/npp_radiation_monitor/data/sensor

Файл RadiationSensor.kt

```
package com.example.npp_radiation_monitor.data.sensor
import android.os.Build
import androidx.annotation.RequiresApi
import com.example.npp_radiation_monitor.data.model.RadiationReading
```

```
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
import kotlin.random.Random

interface RadiationSensor {
    fun getReading(id: Int): RadiationReading
}

class SimulatedRadiationSensor : RadiationSensor {
    @RequiresApi(Build.VERSION_CODES.O)
    override fun getReading(id: Int): RadiationReading {
        val value = Random.nextFloat() * 1.0f // Значення від 0.00 до
1.00 мкЗв/год
        val time =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
        return RadiationReading(
            id = id,
            value = value,
            timestamp = time
        )
    }
}
```

Тека com/example/npp_radiation_monitor/data

Файл RadiationManager.kt

```
package com.example.npp_radiation_monitor.data
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.data.sensor.RadiationSensor
import
com.example.npp_radiation_monitor.data.sensor.SimulatedRadiationSensor

class RadiationManager(
    private val sensor: RadiationSensor = SimulatedRadiationSensor()
) {
    private var nextId = 1

    fun createInitialReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }

    fun generateNextReading(): RadiationReading {
        return sensor.getReading(nextId++)
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл HistroyItem.kt

```
package com.example.npp_radiation_monitor.ui.components
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

@Composable
fun HistoryItem(reading: RadiationReading) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(text = "Замір #${reading.id}", style =
MaterialTheme.typography.labelMedium)
                Text(text = reading.timestamp, style =
MaterialTheme.typography.bodySmall)
            }
            Text(
                text = String.format(locale = Locale.UK, "%.2f мкЗв /
год", reading.value),
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold,
                color = reading.status.color
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл StatusCard.kt

```
package com.example.npp_radiation_monitor.ui.components
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.npp_radiation_monitor.data.model.RadiationReading
import java.util.Locale

@Composable
fun StatusCard(
    reading: RadiationReading,
    modifier: Modifier = Modifier
) {
    val animatedRadiation by animateFloatAsState(targetValue =
reading.value)
    val animatedColor by animateColorAsState(targetValue =
reading.status.color)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .height(180.dp),
        colors = CardDefaults.cardColors(containerColor = animatedColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "Статус: ${reading.status.title}",
                color = Color.White,
```

```
                style = MaterialTheme.typography.titleMedium
            )
            Text(
                text = String.format(locale = Locale.UK, "%.2f",
animatedRadiation),
                color = Color.White,
                fontSize = 56.sp,
                fontWeight = FontWeight.Black
            )
            Text(
                text = "мкЗв/год",
                color = Color.White,
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/screens

Файл RadiationScreen.kt

```
package com.example.npp_radiation_monitor.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ExitToApp
import androidx.compose.material.icons.filled.AccountCircle
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
```

```
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.npp_radiation_monitor.data.RadiationManager
import com.example.npp_radiation_monitor.data.model.RadiationReading
import com.example.npp_radiation_monitor.ui.components.AuthDialog
import com.example.npp_radiation_monitor.ui.components.HistoryItem
import com.example.npp_radiation_monitor.ui.components.StatusCard

@Composable
fun RadiationScreen(
    radiationManager: RadiationManager = remember { RadiationManager() }
) {
    var currentReading by remember
{ mutableStateOf(radiationManager.createInitialReading()) }
    val history = remember { mutableStateListOf<RadiationReading>() }

    // Стан для роботи з авторизацією
    var showAuthDialog by remember { mutableStateOf(false) }
    var operatorEmail by remember { mutableStateOf<String?>(null) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // --- ВЕРХНЯ ШАПКА З АВТОРИЗАЦІЄЮ ---
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    imageVector = Icons.Default.AccountCircle,
                    contentDescription = "Профіль",
                    tint = MaterialTheme.colorScheme.primary
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text(
                    text = if (operatorEmail != null) "Оператор:
$operatorEmail" else "Статус: Гість",
                    style = MaterialTheme.typography.bodyMedium,
                    fontWeight = FontWeight.SemiBold
                )
            }
```

```
            if (operatorEmail == null) {
                OutlinedButton(onClick = { showAuthDialog = true }) {
                    Text("Увійти")
                }
            } else {
                TextButton(onClick = { operatorEmail = null }) {
                    Icon(Icons.AutoMirrored.Filled.ExitToApp,
contentDescription = null)
                    Spacer(modifier = Modifier.width(4.dp))
                    Text("Вийти")
                }
            }
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 12.dp))

        // --- ЗАГОЛОВОК СТАНЦІЇ ---
        Text(
            text = "Моніторинг АЕС",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        // --- КАРТКА СТАНУ ---
        StatusCard(reading = currentReading)

        Spacer(modifier = Modifier.height(12.dp))

        // --- КНОПКА ОНОВЛЕННЯ ---
        Button(
            onClick = {
                val newReading = radiationManager.generateNextReading()
                currentReading = newReading
                history.add(0, newReading)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = MaterialTheme.shapes.large
        ) {
            Icon(Icons.Default.Warning, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("Оновити показники датчика")
        }

        Spacer(modifier = Modifier.height(16.dp))

        // --- ІСТОРІЯ ---
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.Start
        ) {
```

```
            Text(
                text = "Історія вимірювань",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.SemiBold
            )
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))

        LazyColumn(
            modifier = Modifier.fillMaxWidth(),
            contentPadding = PaddingValues(bottom = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(history) { reading ->
                HistoryItem(reading = reading)
            }
        }
    }

    // --- ПОКАЗ СПЛИВАЮЧОГО ВІКНА АВТОРИЗАЦІЇ ---
    if (showAuthDialog) {
        AuthDialog(
            onDismissRequest = { showAuthDialog = false },
            onSuccess = { email ->
                operatorEmail = email
                showAuthDialog = false
            }
        )
    }
}
```

Тека com/example/npp_radiation_monitor

Файл MainActivity.kt

```
package com.example.npp_radiation_monitor
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.material3.Surface
import com.example.npp_radiation_monitor.ui.screens.RadiationScreen
import
com.example.npp_radiation_monitor.ui.theme.NPP_Radiation_MonitorTheme

class MainActivity : ComponentActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 enableEdgeToEdge()
 setContent {
```

```
            NPP_Radiation_MonitorTheme {
                Surface {
                    RadiationScreen()
                }
            }
        }
    }
}
```

Тека com/example/npp_radiation_monitor/ui/components

Файл AuthDialog.kt

```
package com.example.npp_radiation_monitor.ui.components

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp

fun checkErrors(email: String, password: String): String? {
    return when {
        email.isBlank() -> "Заповніть електронну пошту"
        password.length < 6 -> "Пароль повинен містити більше 6 символів"
```

```
        else -> null
    }
}

@Composable
fun Errors(error: String?) {
    if (error == null) return
    if (error.isNotEmpty()) {
        Text(
            text = error,
            color = Color.Red,
            fontWeight = FontWeight.Bold
        )
    }
}

@Composable
fun AuthDialog(
    onDismissRequest: () -> Unit,
    onSuccess: (operatorName: String) -> Unit
) {
    // Перемикач режимів: false = вхід, true = реєстрація
    var isRegisterMode by remember { mutableStateOf(false) }

    // Поля введення
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var name by remember { mutableStateOf("") }
    var passwordVisible by remember { mutableStateOf(false) }

    // Булева змінна, яка каже, чи показувати помилку, чи ні
    var showError by remember { mutableStateOf(false) }

    AlertDialog(
        onDismissRequest = onDismissRequest,
        title = {
            Text(
                text = if (isRegisterMode) "Реєстрація оператора АЕС"
else "Вхід у систему АЕС",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold
            )
        },
        text = {
            Column(modifier = Modifier.fillMaxWidth()) {
                // Поле імені (показується тільки при реєстрації)
                if (isRegisterMode) {
                    OutlinedTextField(
                        value = name,
                        onValueChange = { name = it; showError = false },
                        label = { Text("ПІБ Оператора") },
```

```
                        leadingIcon = { Icon(Icons.Default.Person,
contentDescription = null) },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                }

                // Поле email
                OutlinedTextField(
                    value = email,
                    onValueChange = { email = it; showError = false },
                    label = { Text("Електронна пошта") },
                    leadingIcon = { Icon(Icons.Default.Email,
contentDescription = null) },
                    keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Email),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Spacer(modifier = Modifier.height(8.dp))

                // Поле password
                OutlinedTextField(
                    value = password,
                    onValueChange = { password = it; showError = false },
                    label = { Text("Пароль") },
                    leadingIcon = { Icon(Icons.Default.Lock,
contentDescription = null) },
                    trailingIcon = {
                        IconButton(onClick = { passwordVisible = !
passwordVisible }) {
                            Icon(
                                imageVector = if (passwordVisible)
Icons.Default.Visibility else Icons.Default.VisibilityOff,
                                contentDescription = null
                            )
                        }
                    },
                    visualTransformation = if (passwordVisible)
VisualTransformation.None else PasswordVisualTransformation(),
                    keyboardOptions = KeyboardOptions(keyboardType =
KeyboardType.Password),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Spacer(modifier = Modifier.height(8.dp))

                if (showError) Errors(checkErrors(email, password))
```

```
                Spacer(modifier=Modifier.height(8.dp))

                // Кнопка перемикання вхід / реєстрація
                TextButton(
                    onClick = { isRegisterMode = !isRegisterMode;
showError = false },
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text(
                        text = if (isRegisterMode) "Вже є акаунт? Увійти"
else "Немає акаунту? Зареєструватися"
                    )
                }
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    // Перевіряємо, чи є помилки
                    if (checkErrors(email, password) != null) {
                        showError = true
                    } else {
                        onSuccess(email)
                    }
                }
            ) {
                Text(if (isRegisterMode) "Зареєструватися" else "Увійти")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismissRequest) {
                Text("Скасувати")
            }
        }
    )
}
```